Лекция 2. Функции потерь. Регуляризация

Введение

Ранее мы уже научились представлять объекты в данных как некоторые вектора. После этого ввели модель и показали, что модель линейной регрессии a можно описать как скалярное произведение вектора весов на вектор признаков:

$$a : \mathbb{X} \to \mathbb{Y}$$

$$a(x) = \langle w, x \rangle$$

(при условии, что первый признак — единичный).

Мы рассмотрели самые основы, не вдаваясь в более сложную систему, которая за всем этим стоит. Естественно, мы изучили далеко не все аспекты такого сложного процесса, как *обучения* модели. В частности, непонятными остаются следующие пункты:

— Неужели подбор весов происходит перебором? Можно ли сделать это как-то оптимальнее?

— В какой момент мы понимаем, что веса оптимальные?

— Как мы именно считаем отклонение предсказаний от реальных ответов?

— ...

Целью наших последующих занятий является постепенный ответ на эти вопросы, тем самым собирая более полную картину происходящего и постепенно снимая занавес с этого таинственного процесса обучения модели.

Функции потерь

Для начала надо понимать, а как мы хотим измерять отклонение предсказаний нашей обученной модели от реальных ответов? Ранее мы это определяли, как сумма расстояний каждой конкретной точки (объекта) до построенной прямой (или гиперплоскости). Осталось дело за малым — формализовать наши выводы. С точки зрения геометрии, конечно, это и звучит легко, однако на практике встречаются разные подходы к оцениванию ошибок. Почему это так и почему не хватает какого-то одного подхода — мы рассмотрим далее.

Итак, под *функцией потерь* (или функционалом потерь) мы понимаем некоторую функцию, которая принимает на вход модель, реальные ответы, и на этой основе считает какую-то *ошибку*, т.е. это самое отклонение. Как уже и упоминалось, таких функций далеко не одна, и даже не 10, поэтому постепенно будем проходиться по наиболее популярным и используемым.

MSE, Mean Squared Error

Первая на очереди — средняя квадратичная ошибка (или MSE, или Mean Squared Error). Интуиция: давайте будем считать разницу между реальным ответом и предсказанным, и возводить её в квадрат.

Тогда ошибка на одном объекте считается так:

$$Q(a, y) = (a(x) - y)^2$$

т.е. выход модели на этом объекте минус реальный ответ на этом объекте.

Тогда для всей выборки:

$$MSE = \frac{1}{\ell} \sum_{i=1}^{\ell} (a(x_i) - y_i)^2$$

Интуиция заключается в том, чтобы измерять среднюю величину ошибок модели. То есть мы считаем сумму всех таких ошибок, и делим их на количество объектов.

Ну и вот, казалось бы, мы написали прекрасный функционал потерь, почему бы просто не использовать его везде? Давайте порассуждаем в контексте задачи предсказания цены квартиры: мы вычитаем из предсказанной стоимости реальную, возводим в квадрат и получаем... рубли в квадрате? Я думаю мало кто

сможет объяснить, что такое рубли в квадрате, и своему заказчику тем более.

RMSE, Root Mean Squared Error

Интуиция простая донельзя — давайте просто возьмём корень из MSE:

$$RMSE = \sqrt{MSE}$$

Вот мы и решили ранее описанную проблему — ошибка теперь в рублях, теперь то точно всё хорошо... ведь всё хорошо?

Дело в том, что MSE прекрасно подходит для сравнения двух разных построенных моделей, однако по этому значению тяжело сказать, насколько хороша одна конкретная модель. Давайте предположим, что в наших данных целевая переменная $y \in [0, 1]$. Если MSE выдаёт значение 10, можно догадаться, что это огромная ошибка. При этом если целевая переменная $y \in [10000, 100000]$, то то же значение MSE выглядит очень даже неплохо.

То есть, MSE измеряется в квадратах единиц исходной переменной, поэтому она чувствительна к масштабу. Это означает, что при изменении диапазона целевой переменной та же ошибка будет оцениваться по-разному, что может затруднить интерпретацию в некоторых моментах.

Коэффициент детерминации

Решая вышепоставленную проблему, напрашивается решение: давайте как-нибудь сделаем так, чтобы MSE лежал в интервале $[0, 1]$, вне зависимости от масштаба целевой переменной. Так появился *коэффициент детерминации*, который выглядит довольно страшно:

$$R^2 = 1 - \frac{\sum_{i=1}^{\ell}(a(x_i) - y_i)^2}{\sum_{i=1}^{\ell}(y_i - \overline{y})^2}$$

На самом деле, в числителе и в знаменателе должен быть коэффициент $\frac{1}{\ell}$, но именно поэтому он и сократился. Тогда если приглядеться, то в числителе — уже знакомый MSE, а в знаменателе пока какое-то нечто. Тут $\overline{y}$ обозначает среднее значение целевой переменной. Просто представьте, что в знаменателе тоже MSE модели, которая при любом раскладе выдаёт среднее значение целевой переменной. При такой математике получается нормализованное (т.е. в диапазоне $[0, 1]$) значение MSE. Чем ближе результат коэффициента детерминации к 1, тем

считается лучше. Иногда в литературе коэффициент детерминации называют *долей объяснённой дисперсии*.

Лирическое отступление

Сейчас мы игнорировали одну большую проблему — понятие выбросов в данных. Мы проиллюстрируем это подробнее чуть позже, но проблема заключается в том, что наша модель выдаст какое-то предсказание на выбросе, разница возведётся в квадрат и получится гигантское значение. По тому же коэффициенту детерминации, например, очень сложно понять, что выброс нам что-то испортил. Если забегать сильно вперёд, то по значениям этих функционалов в дальнейшем будут обновляться веса модели — именно поэтому это является сильной проблемой. Наша модель может сильно испортиться даже из за одного выброса. Эту проблему надо как-то проблему.

MAE, Mean Absolute Error

Ранее описанная проблема характеризуется как *неустойчивость к выбросам* в данных. MAE является наиболее устойчивым функционалом потерь, в чём мы убедимся позже. Давайте посмотрим на формулу:

$$MAE = \frac{1}{\ell}\sum_{i=1}^{\ell}|a(x_i) - y_i|$$

Здесь уже больше устойчивости к выбросам. Самое время проиллюстрировать это на примере:

Ответ	Предсказание	MAE	MSE
1	2	1	1
2	3	1	1
100	3	97	9409
4	4	0	0

$$MSE \approx 3137$$

$$MAE \approx 33$$

MAE отдаёт меньший приоритет выбросам, что делает эту метрику наиболее стабильной.

Давайте теперь подумаем: что если нам не столь важно предсказать прям значение ответа, а скажем, просто его порядок? Например в задаче предсказания расстояния между галактиками — будет не сильно страшно в текущих реалиях, если мы ошибёмся на пару сотен световых лет.

MSLE, Mean Squared Logarithmic Error

Когда речь заходит о порядке какой-то величины, обычно в этом же контексте упоминаются логарифмы. Действительно, давайте просто навесим логарифм:

$$MSLE = \frac{1}{\ell}\sum_{i=1}^{\ell}(\log(a(x_i)+1) - \log(y+1))^2$$

Надеюсь не трудно догадаться, что такое RMSLE:

$$RMSLE = \sqrt{MSLE}$$

Давайте пойдём ещё дальше. Теперь мы хотим получать не просто какое-то число, которое интерпретируется как какая-то там ошибка, а хотим получать какую-то относительную ошибку. Например сказать «ошибка 50%» звучит лучше, чем «MSE = 123.41286...».

(S)MAPE, (Symmetric) Mean Absolute Percentage Error

Формула:

$$MAPE = \frac{1}{\ell}\sum_{i=1}^{\ell}\frac{|y_i - a(x_i)|}{y_i} * 100\%$$

Однако у MAPE есть проблема, связанная с несимметричностью данной функции. Давайте допустим, что ответ на объекте равен 1, а прогнозы нашей модели неотрицательные. При *недопрогнозе* (когда модель выдала значение меньше, чем ответ) максимальная ошибка может быть 100%:

$$\frac{|1-0|}{1} * 100\% = 100\%$$

В случае *перепрогноза* (когда модель выдала значение больше, чем ответ), мак-

симальная ошибка неограничена:

$$\frac{|1-10|}{1} * 100\% = 900\%$$

Эту проблему решает SMAPE:

$$SMAPE = \frac{1}{\ell}\sum_{i=1}^{\ell}\frac{|y_i - a(x_i)|}{(|y_i| + |a(x_i)|)/2}$$

Функционалов потерь, конечно, существует достаточно много, но мы остановимся лишь на этих базовых. Главное, что нужно из всего этого извлечь — **с помощью функционала ошибок мы задаём требования к нашей модели**.

Вернёмся к задаче обучения модели. В процессе подбора параметров и подсчёта ошибок, понятное дело мы хотим, чтобы выбранный функционал был как можно меньше. Формализуя (на примере MSE):

$$\frac{1}{\ell}\sum_{i=1}^{\ell}(a(x_i) - y_i))^2 \to \min_w$$

Формально, это и есть процесс обучения модели. Мы пока не будем затрагивать процесс подбора этих параметров w, потому что это сложная тема для отдельной лекции.

Регуляризация: зачем и как

Веса модели могут принимать любые значения. Считаем логичным, что чем больше вес у определённого признака, тем этот признак важнее для предсказания. И наоборот, чем вес меньше, тем наименее полезен. Звучит логично, но иногда бывают ситуации, когда слишком большие или слишком малые веса приводят к неожиданному поведению модели. Посмотрите на следующее изображение:

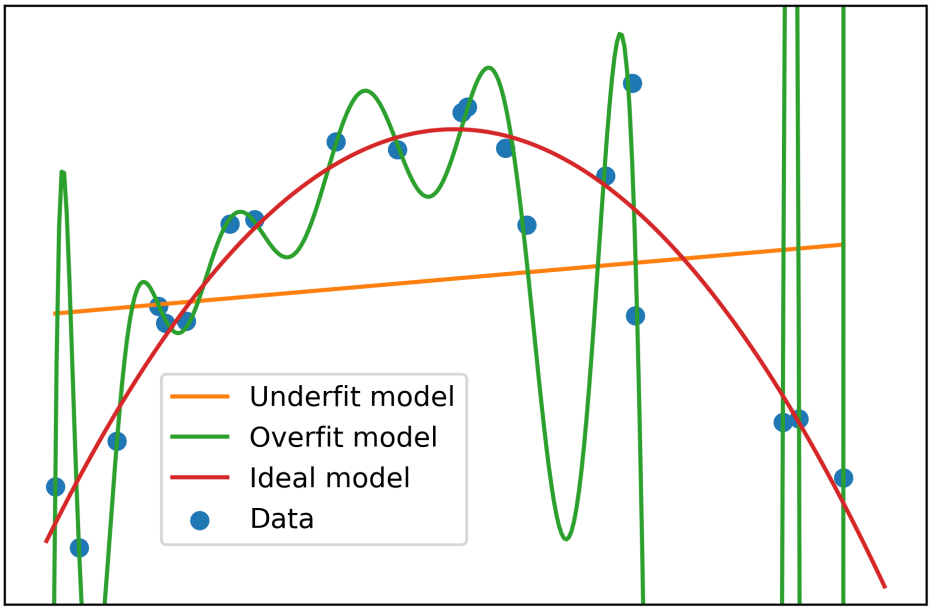


Рисунок 0.1 – Поведение модели в зависимости от весов

Красная модель является в некотором смысле идеальной. В каждой задаче мы предполагаем, что существует функция, которая идеально описывает данные, и наша задача — как можно ближе восстановить эту функцию.

Оранжевая модель не справляется с задачей описания представленных данных. Не вдаваясь в подробности, такой случай называется *недообучением* (underfitting). Модель так и не научилась описывать данные — это может быть связанно с, например, маленьким количеством представленных данных.

Зелёная модель слишком хорошо справляется с обучающими примерами, буквально проходя через каждую из точек. Проблема заключается в том, что при поступлении новых данных, эта модель, многовероятно, будет предсказывать очень плохо. А зачем нам модель, если она не сможет предсказывать что-то на новых данных? Такое поведение модели называется *переобучением* overfitting, и это является наиболее насущной проблемой, с которой нужно бороться в любом

случае.

Есть чисто эмпирическое наблюдение, которое заключается в том, что если у модели по модулю большие веса — это означает, что скорее всего она переобучена.

Мы рассмотрим метод, который в литературе обычно называют $\mathrm{weight\ decay}$ (уменьшение весов), и его идея донельзя проста: давайте просто к функционалу потерь приблавлять эти же веса, с некоторым коэффициентом α. Этот коэффициент будет выражать степень силы регуляризации. Для MSE:

$$\frac{1}{\ell}\sum_{i=1}^{\ell}(a(x_i)-y_i)^2+\alpha||w||_1$$

$||w||_1$ — это умное обозначение всех элементов вектора. Это называют L_1 регуляризацией.

Теперь нам надо минимизировать не только отклонение предсказания от ответа, но ещё и сумму весов, домноженную на какой-то коэффициент ($\alpha \in [0,1]$), и нашей модели теперь непозволительно делать какие-то большие значения весов.

Есть ещё один вид регуляризации. Прежде чем задаваться вопросами «а зачем», давайте обсудим её. Идея та же, но теперь делаем не просто сумму, а сумму квадратов весов:

$$\frac{1}{\ell}\sum_{i=1}^{\ell}(a(x_i)-y_i)^2+\alpha||w||_2^2$$

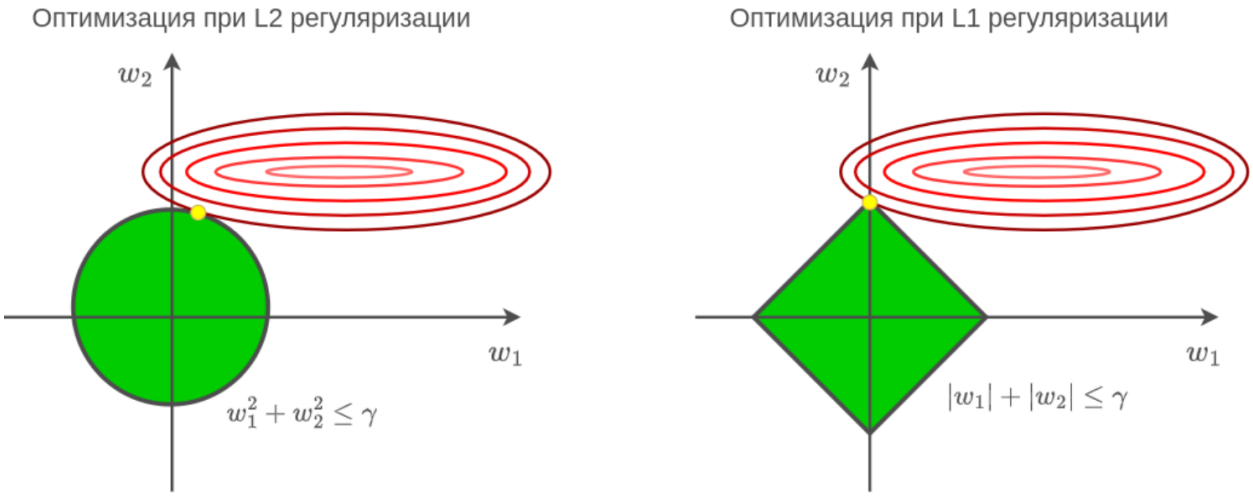
Это же чудо называется L_2 регуляризацей. Интуитивно, чем больше вес — тем сильнее штраф.

Ну и теперь осталось обсудить самое главное — зачем нам 2 типа регуляризации?

Дело в том, что L_1 обладает интересной способностью *занулять* некоторые веса, а соответственно и признаки. Существует множество способов описать такое поведение, но давайте взглянем на наиболее интуитивно понятное.

Прежде чем приступать, надо объяснить, что такое линии уровня. Представьте себе сложную трёхмерную функцию, на которую мы смотрим сверху так, чтобы она выглядела как двумерная — это и есть линии уровня. С помощью та-

кого приёма мы можем проецировать многомерные функции в пространство меньшей размерности. Взглянем на следующее изображение:



Красная проекция — это линии уровня для MSE. Рассмотрим подробнее левое изображение. Зелёный график — это интерпретация L_2 нормы для двух весов. Действительно, получится $w_1^2 + w_2^2$, что есть уравнение круга с центром в точке $(0, 0)$. Пересечение круга с линиями уровнями означает, что MSE достигает своего минимального значения именно в этой точке пересечения. Заметим, что при этом вес ненулевой.

Теперь рассмотрим $L_1 = |w_1| + |w_2|$. Это есть уравнение ромба, что мы и видим на рисунке справа. Пересечение в большинстве случаев происходит в угловой точке ромба, где один из весов (в данном случае w_2) равняется 0.

Выходит, что в некоторых ситуациях, с точки зрения L_1 оптимально будет занулять некоторые веса.